Systolic Arrays:
From 1970s VLSI to Modern AI Matrix Engines

Mahesha S
Compiler Engineer
mahesha.shivamallappa@gmail.com

Abstract

Systolic arrays were proposed by Kung and Leiserson in 1978 as a special-purpose VLSI architecture for exploiting data reuse in signal-processing kernels such as convolution and banded matrix–vector multiplication [8]. The idea receded from mainstream computer architecture as general-purpose processors improved rapidly through the following decades, only to re-emerge in modern AI hardware, where regular dataflow, local communication, and operand reuse again became valuable architectural principles [6]. This paper gives a self-contained, example-driven account of the systolic array, from its 1978 origins to its role in modern AI matrix engines. We build up the architecture concretely: starting from a small fully-connected neural-network layer, we show how a weight-stationary systolic mapping realizes the layer's arithmetic, derive the cycle-accurate timing—skewing, wavefront propagation, and fill/drain behaviour—that makes nearest-neighbour communication sufficient, and extend the single-vector case to pipelined matrix–matrix multiplication. We then use TPU v1 as a concrete architectural case study, examining its matrix unit and host–accelerator programming model and clarifying in what sense a domain-specific accelerator can be programmable without being general-purpose. We close by explaining, via a roofline-style argument, how operand reuse and reduced data movement can improve the effective arithmetic intensity and energy efficiency of suitable workloads [5, 13], and by situating systolic arrays within the broader family of specialized matrix-multiply architectures in modern AI hardware, distinguishing explicitly systolic designs, such as AWS's Trainium and Inferentia2, from related but architecturally distinct engines such as NVIDIA's Tensor Cores and Apple's Neural Engine, and from variants such as the row-stationary mapping used by academic accelerators such as Eyeriss [1–3, 9, 12]. The treatment throughout is architectural and conceptual rather than implementation-level: we stop at processing elements, dataflow, and timing, and do not address circuit design, physical layout, or fabrication.

Keywords: systolic arrays, Kung–Leiserson, VLSI, matrix multiplication, weight-stationary dataflow, wavefront, operand reuse, arithmetic intensity, roofline model, Tensor Processing Unit (TPU), domain-specific architecture, AI accelerators.

Contents

1 Historical Origins: Kung and Leiserson, 1978–1982 **3**
- 1.1 The heart-pump metaphor . . . 3
- 1.2 Why the idea left the mainstream . . . 4
- 1.3 The rediscovery . . . 4
- 1.4 Scope of this paper . . . 4

2 Basic Structure **4**
- 2.1 Anatomy of one PE . . . 7
- 2.2 Local interconnect, not a crossbar or bus . . . 8

3 Execution Model **8**
- 3.1 Skewing creates the wavefront . . . 8
- 3.2 Fill, steady state, and drain . . . 8
- 3.3 Dataflow variants: what is held stationary . . . 10

4 The Computational Idea: Mapping Matrix Multiply Onto the Array **10**
- 4.1 Why this saves memory traffic . . . 13
- 4.2 Beyond square GEMM: convolution and batching . . . 13

5 Modern Incarnation: The TPU's Matrix Multiply Unit **13**
- 5.1 What sits around the array . . . 13
- 5.2 The programming model: how a TPU actually gets used . . . 15
- 5.3 What changed since 1978, and what didn't . . . 15

6 Beyond Google: The Idea Generalized **16**

7 Why the Idea Fits Today's Deep Learning Workloads **17**
- 7.1 High, and controllable, arithmetic intensity . . . 17
- 7.2 Energy: data movement, not arithmetic, dominates . . . 17
- 7.3 Control overhead amortized over huge, regular tensors . . . 18
- 7.4 Where the fit is imperfect . . . 18

8 Summary: 1978 vs. Today, Side by Side **19**

1 Historical Origins: Kung and Leiserson, 1978–1982

The systolic array was not invented for neural networks. It was proposed in the late 1970s by H. T. Kung and Charles Leiserson at Carnegie Mellon, in the middle of the first VLSI (Very-Large-Scale Integration) revolution, when chip designers were realizing that the number of transistors they could pack onto a die was growing far faster than the number of pins that could get data on and off it.

Their 1978 paper [8] (circulated as a CMU technical report, and published more widely in 1979–82 as "Systolic Arrays for VLSI" and in the book *Introduction to VLSI Systems* with Carver Mead and Lynn Conway) posed a specific engineering problem: how do you design a special-purpose chip so that once data is pulled in from memory, it is reused by as many arithmetic units as possible before it has to leave the chip again? A central concern was the imbalance between abundant on-chip computation and comparatively scarce off-chip I/O bandwidth — an early form of the data-movement problem that later became associated with the "memory wall."

1.1 The heart-pump metaphor

Kung and Leiserson named the idea after the systole of a heartbeat: blood (data) is pumped rhythmically through a network of chambers (processing elements, or PEs), and every chamber does a small amount of work on the data as it passes through, in lock-step with a global clock. No chamber ever reaches far away for data — it only exchanges with its immediate neighbours.

This same contrast — conventional execution versus systolic execution — is the clearest way to see what the mesh actually buys you.

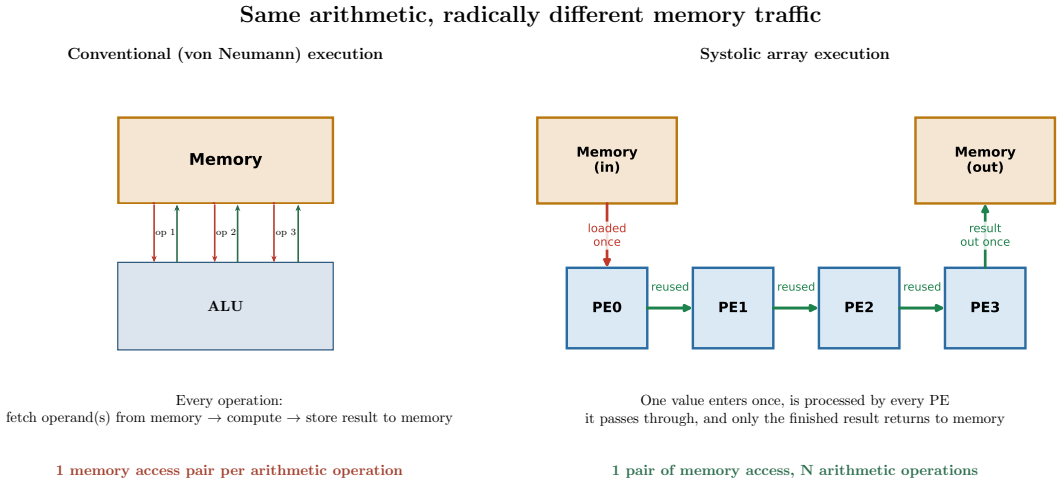


Figure 1: In a conventional design, memory is touched on every single operation. In a systolic array, a value is fetched from memory once and then reused by every PE it flows past before the finished result is written back — the whole point of the mesh is to turn memory accesses into a one-time cost instead of a per-operation cost.

This is the single defining property of a systolic array:

- **Regular, local interconnect** — every PE talks only to its nearest neighbours.
- **Synchronous, rhythmic data movement** — data advances one hop per clock tick.
- **Reuse** — each datum is reused by several PEs before leaving the array, instead of one memory fetch feeding one arithmetic operation.

The original applications were narrow and hardware-flavoured: convolution, FIR filtering, banded matrix–vector and matrix–matrix multiplication, pattern matching, and sorting networks — problems whose data-dependency graphs are themselves regular grids or wavefronts, and therefore map naturally onto a physical grid of ALUs.

1.2 Why the idea left the mainstream

Through the 1980s and 90s, general-purpose CPUs rode Moore's law and Dennard scaling so effectively that building a custom systolic chip for one narrow kernel was rarely worth the design cost — by the time the chip taped out, a faster general-purpose CPU had usually shipped anyway. Systolic ideas survived mainly inside specific signal-processing ASICs (Application-Specific Integrated Circuits) and in the academic literature on parallel algorithms, rather than as a mainstream computer-architecture technique.

1.3 The rediscovery

By the 2010s, two developments made the idea attractive again: deep neural networks (DNNs) made dense matrix multiplication (GEMM) and convolution dominant workloads, and the growing cost of data movement relative to arithmetic made operand reuse increasingly valuable. That is the same broad imbalance the systolic array was built to address in 1978. Modern computer-architecture texts — most notably the treatment of Google's Tensor Processing Unit in Hennessy and Patterson's discussion of domain-specific architectures [4] — describe the TPU as an industrial-scale revival of the Kung–Leiserson idea: TPU v1 used 8-bit matrix multiplication, while later generations adopted additional numerical formats such as bfloat16 for modern machine-learning workloads [7].

1.4 Scope of this paper

Scope

This paper is intentionally a high-level, conceptual treatment of the systolic array: it is aimed at building an accurate mental model of how the mesh is structured, how it is timed, and how it maps onto matrix multiplication and, through that, onto modern deep-learning accelerators. It deliberately stops at the level of processing elements, dataflow, and cycle-accurate timing diagrams. It does *not* descend into circuit-level design, RTL implementation, physical layout and floorplanning, timing closure, or the manufacturing and verification considerations that separate a systolic array described on paper from one that ships in silicon; nor does it provide a comprehensive survey of every systolic or near-systolic accelerator that has been built. Readers seeking that lower-level treatment, or a broader survey of the accelerator design space, should consult the original VLSI-design and computer-architecture literature this paper draws on and cites throughout.

2 Basic Structure

The clearest way into the structure is a concrete example rather than an abstract mesh. Take one of the simplest building blocks in deep learning: a tiny fully-connected layer with 4 input features and 4 output neurons. Every input is connected to every output by a learned weight, so this one small layer has 16 weights in total, and each output is a weighted sum of all four inputs:

$$y_j = \sum_i x_i \times w_{ij}, \tag{1}$$

a basic sum-of-products pattern underlying fully-connected layers and many of the linear transformations used in convolutional and attention-based models.

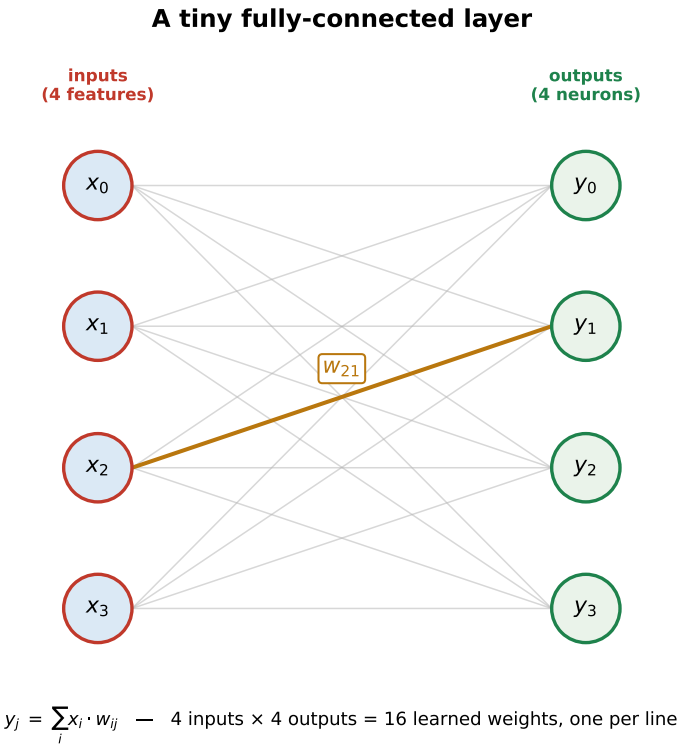


Figure 2: A tiny fully-connected layer: 4 input features, 4 output neurons, 16 weights — the ordinary way this computation is drawn.

This is the picture every deep learning course starts with. The systolic array is simply a different physical layout of the exact same 16 weights and the exact same arithmetic — one multiply–accumulate cell (a PE) per weight, wired into a 2-D mesh instead of drawn as crossing lines (Figure 3).

This picture already settles three things about how the array behaves — one about wiring, one about memory, and one about timing.

- **Locality:** every PE talks only to its nearest neighbours. PE$(2,1)$ exchanges data with PE$(2,0)$, PE$(2,2)$, PE$(1,1)$, and PE$(3,1)$, and with no PE anywhere else in the array.
- **Weight residency:** each weight is loaded once and stays exactly where it is for as long as the layer keeps computing on new inputs. The 16 weights in Figure 2 become 16 fixed residents of the mesh, not 16 values re-fetched from memory on every use — and this generalizes directly: a real layer with 256 inputs and 256 outputs is the same picture, just 256×256 instead of 4×4, which happens to be exactly the size of TPU v1's array, the case study in Section 5 (later TPU generations use different array sizes).
- **Timing:** the horizontal and vertical arrows are not independent — they have to arrive at each PE in step, or the sum computed there is wrong.

That last point is easy to miss in a static picture, so it is worth tracing through concretely. The horizontal arrows only ever carry x_i unchanged, one hop right per clock cycle; nothing is computed on that path. The vertical arrows carry the running partial sum for column j, updated by exactly one PE's product $(x_i \times w_{ij})$ per hop down. For PE(i,j) to compute the right thing, x_i has to land on it in the very same cycle as the partial sum arriving from PE$(i-1,j)$ above it — concretely, x_2 must reach PE$(2,1)$ at the same instant as the partial sum flowing down from PE$(1,1)$. That alignment does not happen by accident; it is engineered by releasing

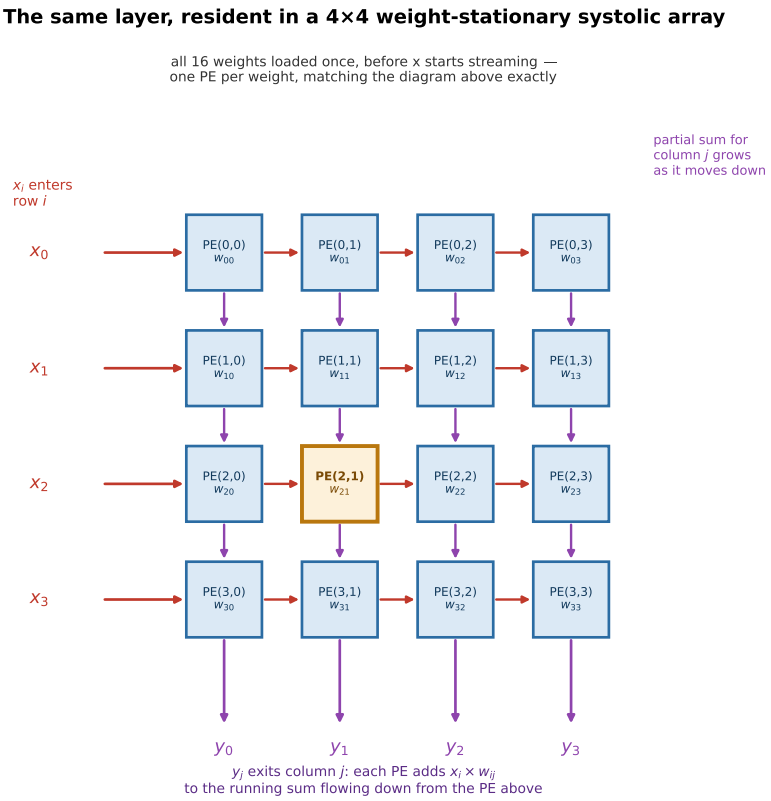


Figure 3: The identical layer, laid out as a 4×4 systolic array. PE(i, j) permanently holds weight w_{ij} — the same w_{21} highlighted in Figure 2 sits in the highlighted PE$(2, 1)$ here. Input x_i enters row i from the left; output y_j accumulates down column j and exits at the bottom.

each row's input one cycle later than the row above it (x_0 at cycle 0, x_1 at cycle 1, x_2 at cycle 2, and so on). With that stagger, x_i reaches PE(i, j) at cycle $i + j$, and the partial sum for column j reaches PE(i, j) at cycle $i + j$ too, having picked up one cycle of delay for every PE it passed through on the way down. The two paths — one delayed by when it was injected, the other delayed by how far it has travelled — land on the same PE at the same instant purely by this construction. Section 3.1 works through exactly how that staggered injection is engineered, and shows it as a moving diagonal wavefront sweeping across the whole array.

2.1 Anatomy of one PE

Each PE is deliberately minimal — that simplicity is what lets thousands of them fit on one die. Take PE$(2, 1)$ from the example above: it permanently holds w_{21}, the single weight connecting input x_2 to output y_1. A typical PE holds:

- A multiply–accumulate (MAC) unit — one multiplier and one adder.
- A small number of registers: one for a held ("stationary") operand, one or two for the operand(s) passing through, and one accumulator register for the running partial sum.
- Pass-through wiring to the PE below and to the PE on the right (or whichever two neighbours the chosen dataflow requires).

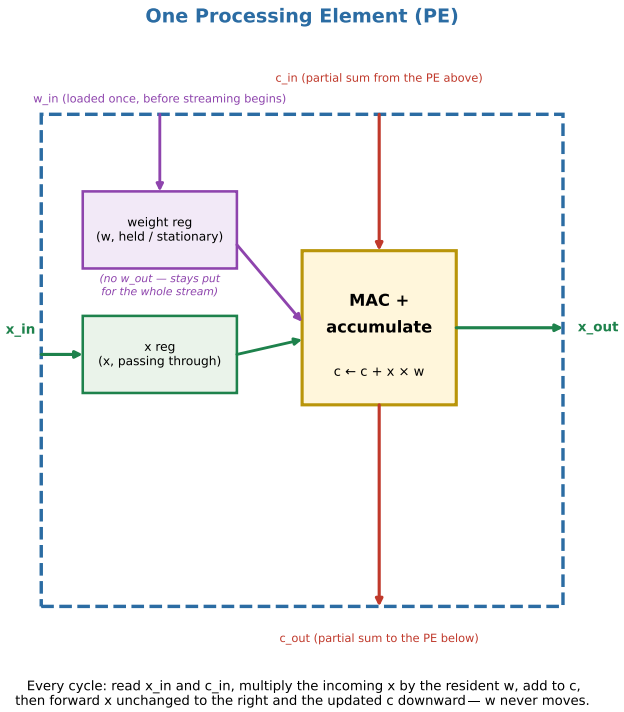


Figure 4: Internals of a single weight-stationary PE — concretely, PE$(2, 1)$ from Figure 3. Its weight register holds w_{21}, loaded once and never moving; x_2 passes through unchanged on its way to PE$(2, 2)$; the partial sum for column 1 is read in from PE$(1, 1)$ above, updated by adding $x_2 \times w_{21}$, and pushed down to PE$(3, 1)$.

There is no instruction fetch, no program counter, and no addressable memory inside a PE. Every PE executes the identical fixed operation — typically

$$c \leftarrow c + x \times w, \qquad x_{\text{out}} \leftarrow x \tag{2}$$

every cycle — w itself only moves during the initial load, then stays put for the weight-stationary case shown above; a fully general PE would also forward w_{out} for dataflows where the weight streams instead.

This is why systolic PEs can be so much smaller and more energy-efficient per MAC than a general-purpose core: they do not fetch or decode an instruction stream, and most of their behaviour is determined by the fixed dataflow and the clock.

2.2 Local interconnect, not a crossbar or bus

Crucially, no PE ever broadcasts to the whole array and no PE reads an arbitrary memory address. Every wire in the array is a short, fixed connection to a neighbour, which is what keeps clock frequency high and wiring energy low even as the array grows to hundreds of PEs on a side — a property that would be impossible with a shared bus or crossbar of the same size.

3 Execution Model

A systolic array is a purely synchronous machine: on every clock edge, each PE simultaneously reads whatever its neighbours handed it last cycle, performs its fixed operation, and hands the results to the next PEs. The interesting part is how data is fed in and drained out so that every PE is always working on correctly aligned operands.

3.1 Skewing creates the wavefront

If every row's input value x_i entered the array in the very same cycle, the operands arriving at different rows would not be aligned with the corresponding partial sums. The standard solution is to skew the inputs: row i's data is injected i cycles later than row 0's. This timing ensures that each PE receives the input value and partial sum belonging to the same computation in the same cycle. The result is a diagonal wavefront of active computation that sweeps across the array.

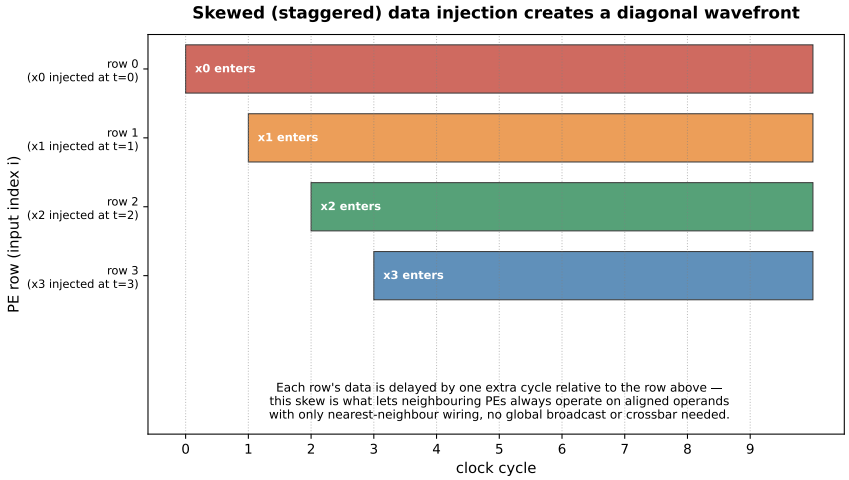


Figure 5: Staggering each row's injection by one extra cycle produces a diagonal wavefront: at any instant, exactly the PEs on one diagonal are doing meaningful work with correctly aligned data.

Figure 5 shows timing — when each row is active — but it is worth seeing the same idea on the mesh itself: "diagonal wavefront" is a literal description of what lights up on the 2-D grid at each instant, not just a figure of speech.

3.2 Fill, steady state, and drain

It is useful to distinguish the behaviour of a single wavefront from that of a continuous stream:

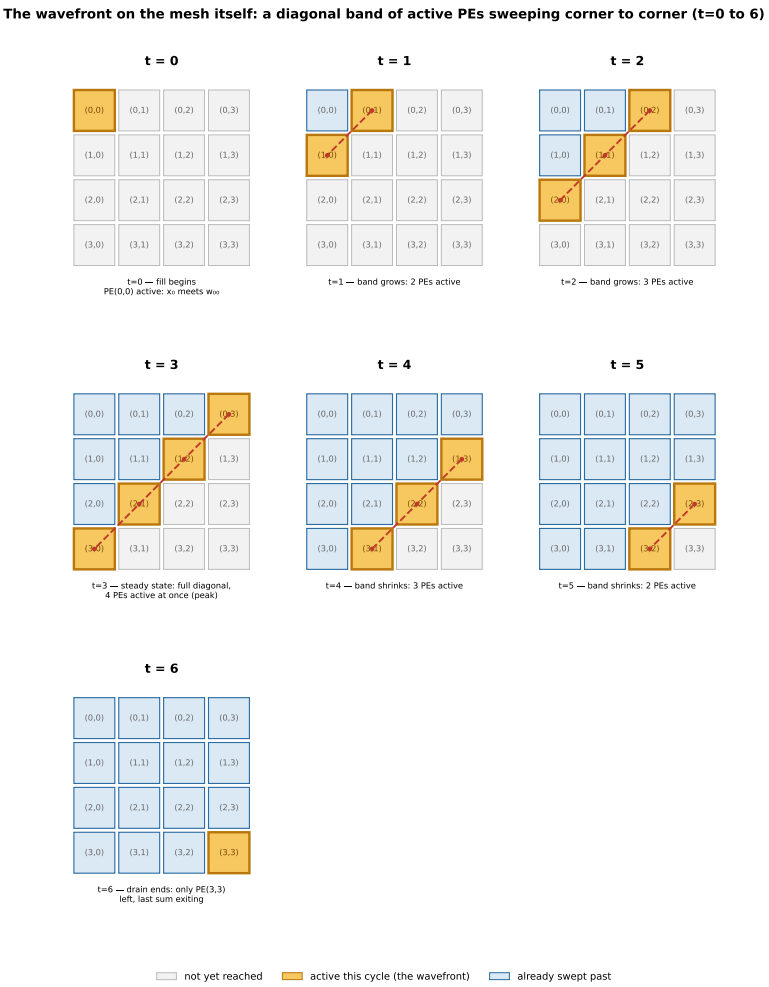


Figure 6: The same wavefront, viewed on the 4×4 mesh from Figure 3, shown for every cycle from $t = 0$ to $t = 6$. At each cycle, exactly the PEs with $i + j = t$ are active (orange) — a diagonal band that enters at PE$(0,0)$, grows one PE wider each cycle to its full width of 4 PEs at $t = 3$, then shrinks back down to a single PE as it exits at PE$(3,3)$.

- **Fill** — the first $\sim N$ cycles, while the first wavefront is entering the array and its active diagonal is still growing.
- **Peak / steady streaming** — for a single vector, the widest wavefront contains only N active PEs, one diagonal at a time. If many vectors or rows are streamed back-to-back, however, several wavefronts occupy the array simultaneously; after the pipeline fills, all N^2 PEs can perform useful MACs every cycle.
- **Drain** — when the input stream stops, the remaining partial sums continue propagating until the final results leave the array.

For a single $N \times N$ matrix–vector operation, the visible wavefront therefore grows and shrinks rather than keeping all N^2 PEs busy. High sustained utilization appears when many vectors are pipelined through the same loaded array. For a full $N \times N$ matrix multiplication, the multiple rows of the left-hand matrix provide exactly such a stream, and the fill/drain overhead becomes increasingly well amortized as more rows, batches, or matrices are processed back-to-back.

3.3 Dataflow variants: what is held stationary

Different systolic designs choose to keep a different operand fixed inside each PE for as long as possible, because whichever operand is stationary is the one you don't have to keep re-reading from memory:

Dataflow	What stays fixed inside each PE	Trade-off
Weight-stationary	A weight value, loaded once per matrix and held for the whole x stream	Best when the same weights are reused across many inputs (large batch / long sequence) — used by Google's TPU v1, the case study in Section 5
Output-stationary	A partial sum, accumulated in place while both operands stream past it	Avoids ever writing partial sums to memory; needs both operand streams skewed
Input-stationary	An input activation is retained locally while several weights or output computations reuse it	Useful when activation reuse is the dominant opportunity
Row-stationary	A convolution-specific hybrid mapping that exploits reuse of inputs, weights, and partial sums across a spatial row	Used by Eyeriss; it should not be treated as simply another name for input-stationary [3, 12]

Table 1: Three simple stationary dataflows, plus the convolution-specific row-stationary mapping. The first three describe which principal operand is retained locally; row-stationary is a hybrid designed to exploit several forms of reuse simultaneously.

4 The Computational Idea: Mapping Matrix Multiply Onto the Array

Mapping matrix multiplication onto a systolic array is a natural fit, because the data-dependency structure of matrix multiplication is itself a regular 2-D grid: computing $C = X \times W$, entry c_{ij} depends on row i of X and column j of W, and adjacent output entries share most of their inputs. A key architectural insight is that this regular sharing pattern can be mapped naturally onto nearest-neighbour propagation on a mesh.

Concretely, for a weight-stationary mapping of an $N \times N$ weight matrix W:

- Before computation starts, W is loaded into the array so that PE(i, j) permanently holds w_{ij}.

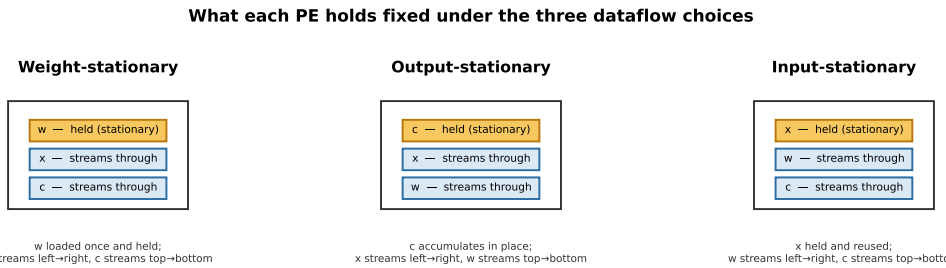


Figure 7: The three simple stationary dataflows illustrated with the same PE abstraction: weight-, output-, and input-stationary. Row-stationary, discussed in Table 1, is a more specialized convolution mapping and is not shown here, since it cannot be captured simply by pinning a different operand in this generic diagram.

- Rows of the input matrix X are streamed in from the left — each row is one x-vector, exactly like the one in Figure 2, just now one of many stacked into a batch — one element per cycle per row, skewed as in Section 3.1.
- At every cycle, PE(i,j) computes $c \leftarrow c + x \times w_{ij}$ using the x value currently passing through it, then forwards that same x value one step to the right, and lets the accumulating partial sum for that column continue toward the bottom.
- When a partial sum reaches the bottom row, it is the finished dot product — one full output element of C — with no intermediate value ever having been written to, or read back from, memory.

Figure 6 showed this wavefront for one x-vector: the case where X has exactly one row, so $M = 1$ and the array performs a vector-times-matrix multiplication. A real X almost always has more than one row — M separate x-vectors, one per sample in a batch or one per token in a sequence.

It is worth being precise about what happens at a single PE when a batch is streaming, since no PE ever receives more than one number at a time. Each row of X is injected exactly the way Section 3.1 describes for a single vector: feature x_0 enters PE$(0,0)$, feature x_1 enters PE$(1,0)$ one cycle later, feature x_2 enters PE$(2,0)$ another cycle after that, and so on — one feature per PE, unchanged from before. What is new for a batch of M rows is only this: PE$(0,0)$ does not sit idle once sample 0's x_0 has passed through it. The very next cycle, PE$(0,0)$ receives sample 1's x_0 instead — a different number, arriving on that same single wire — then sample 2's x_0 the cycle after that, and so on. Concretely, PE$(0,0)$ never receives "row 0 of X"; over four consecutive cycles it receives four separate numbers, one at a time: x_0 from sample 0, then x_0 from sample 1, then x_0 from sample 2, then x_0 from sample 3, each multiplied by the same resident w_{00} and forwarded rightward before the next one arrives. Every other PE in the array is doing the identical thing at the same time, each with its own resident weight and whichever single value currently occupies its diagonal.

Because each row's own wavefront still takes $2N-1$ cycles to fully cross the array, and each row's first feature starts one cycle after the previous row's first feature did, several rows end up in flight inside the array at once — this is what Figure 8 shows.

Figure 8 makes this concrete for the same 4×4 array used throughout this paper, now with X itself a 4×4 matrix: 4 rows of X (4 x-vectors, colour-coded). Each row's first feature begins entering the array one cycle after the previous row's first feature did — not the whole row at once, but one feature per cycle as just described. The panels cover the full computation, cycles $t = 0$ through $t = 9$, so every row's entry, peak, and exit is visible. Rows start one at a time from $t = 0$ to $t = 3$; by $t = 3$ all 4 rows are simultaneously in flight, and they stay simultaneously in flight through $t = 6$; from $t = 7$ onward rows finish draining one at a time — row 0 first, then row 1, then row 2 — until only row 3's last cell remains at $t = 9$, at which point the whole 4×4-by-4×4 multiplication is complete.

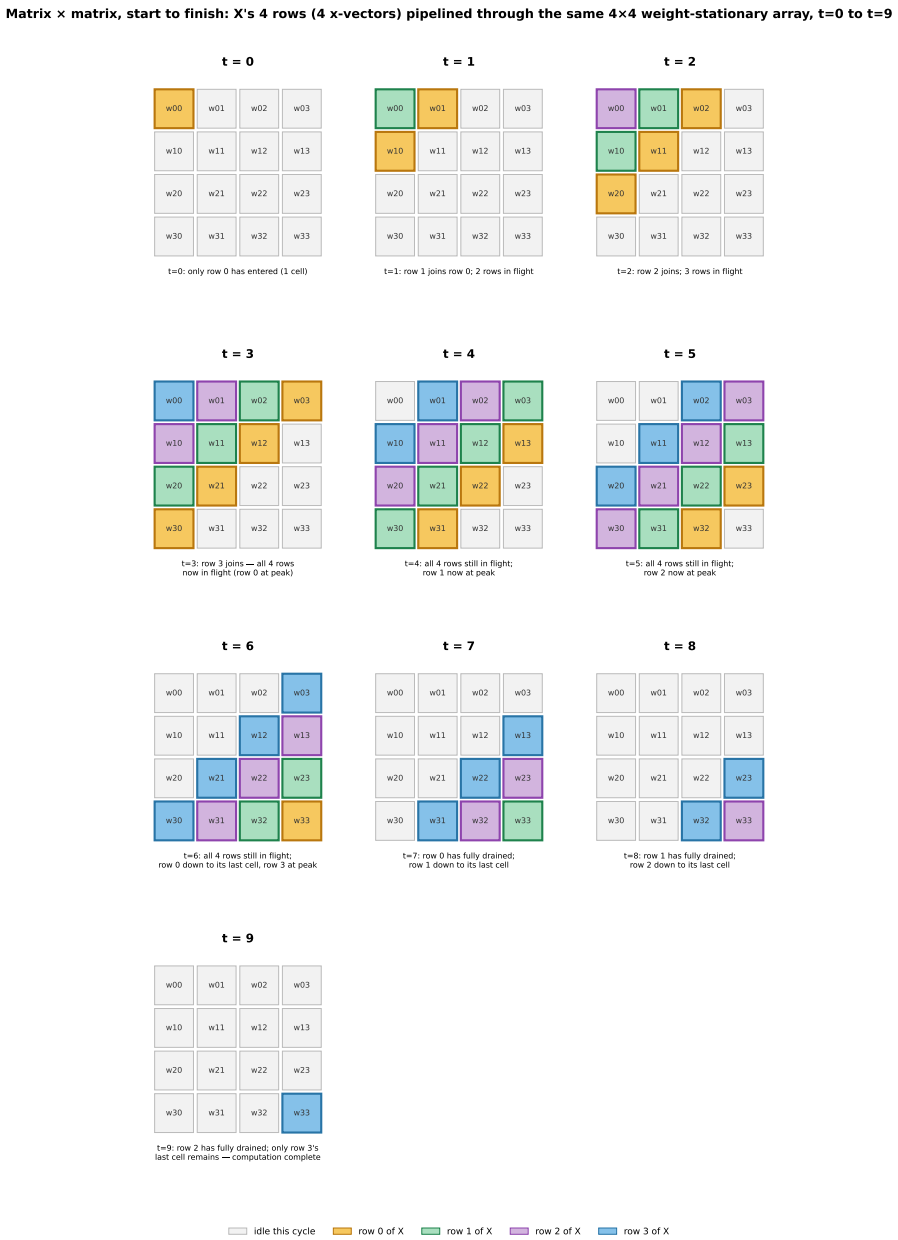


Figure 8: Matrix × matrix on the same 4×4 array as Figures 3 and 6, with X now a 4×4 matrix (4 rows, shown in 4 colours), each row entering one cycle after the previous row. All 10 cycles are shown, $t = 0$ through $t = 9$: rows join one at a time through $t = 3$, all 4 rows are simultaneously in flight from $t = 3$ through $t = 6$, and rows exit one at a time from $t = 7$ through $t = 9$, when the computation finishes.

4.1 Why this saves memory traffic

As an idealized comparison, assume the relevant $N \times N$ tiles can remain resident on-chip while they are processed. A scheme with no reuse would repeatedly fetch operands, leading to $O(N^3)$ operand traffic for an $N \times N$ matrix product. In the systolic mapping, each resident weight is reused across a stream of inputs and each input value is reused as it traverses the array, reducing the external traffic associated with those tiles toward $O(N^2)$ while the arithmetic work remains $O(N^3)$. Real systems also incur traffic for loading and storing tiles, so the exact count depends on tiling and the surrounding memory hierarchy; the architectural point is the same: the array turns repeated arithmetic on the same operands into local reuse rather than repeated off-chip movement.

4.2 Beyond square GEMM: convolution and batching

Many convolution implementations can be expressed as matrix multiplication after transformations such as the classic "im2col" mapping, while others use direct convolution-specific dataflows. In either case, the underlying computation contains large regular collections of multiply–accumulates, which can be mapped onto suitable spatial dataflows. A whole minibatch or a whole sequence of tokens is simply a longer stream of x rows flowing through the same pre-loaded weights, which is precisely what amortizes the fill/drain overhead from Section 3.2 to near zero.

5 Modern Incarnation: The TPU's Matrix Multiply Unit

Google's Tensor Processing Unit, first deployed internally in 2015 and described publicly in the 2017 ISCA paper "In-Datacenter Performance Analysis of a Tensor Processing Unit" [6], is the clearest large-scale, commercially deployed descendant of the Kung–Leiserson array. For concreteness, this section uses TPU v1 as the main architectural case study, because its relatively simple organization exposes the systolic-array idea particularly clearly. At its core sat a 256×256 Matrix Multiply Unit (MXU): 65,536 multiply–accumulate cells arranged as the regular mesh developed earlier in this paper and operated with a weight-stationary dataflow. Later TPU generations evolved substantially around the same broad idea, so details in this section should not be read as a literal description of every current TPU generation.

5.1 What sits around the array

Five blocks surround the MXU, each doing one job in the data path Figure 9 shows:

- **Unified buffer** — on-chip SRAM holding activations, avoiding a DRAM round-trip between layers where possible.
- **Weight FIFO** — feeds weight matrices into the MXU's stationary registers ahead of each matrix multiplication.
- **Accumulators** — collect the partial sums draining out of the array; TPU v1 accumulated results at higher precision than its 8-bit integer inputs [6]. Later generations changed numerical formats and other details.
- **Activation pipeline** — applies nonlinearities, pooling, and normalization to the MXU's output before it becomes the next layer's input.
- **Sequencer** — fetches and executes the instructions that drive all of the above; what those instructions are, and where they come from, is the subject of Section 5.2.

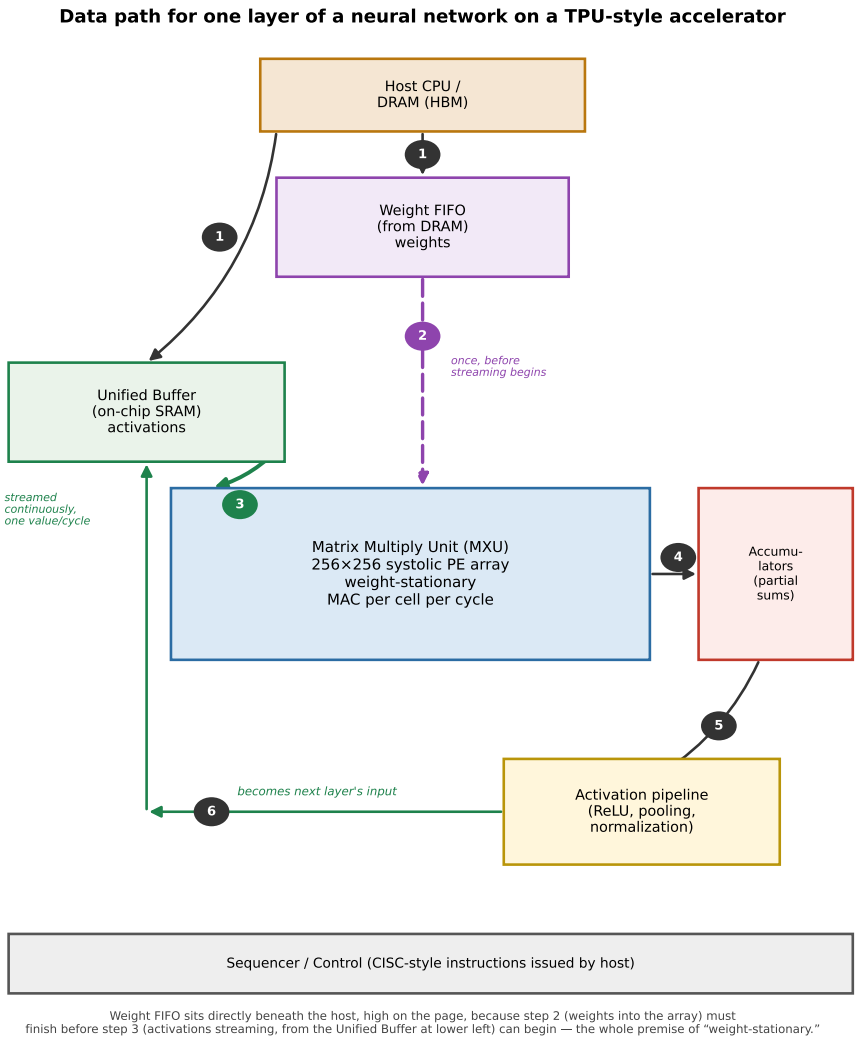


Figure 9: Conceptual TPU-style data path for one layer, numbered in the order data moves. The diagram is intended to explain the architectural roles and ordering of the major blocks rather than reproduce one exact TPU generation. (1) the host-side system supplies weights and activations; (2) weights are loaded into the matrix engine and become resident; (3) activations then stream through the array; (4) finished sums move to the accumulators; (5) an activation path applies subsequent element-wise processing; (6) the result becomes input to a later computation. The ordering is the important point: for a weight-stationary mapping, weights are resident before the corresponding activation stream is consumed.

Terminology: "activations"

The first bullet is worth being precise about. Its contents are called *activations* throughout this paper, but that word is only exactly right from the second layer of a network onward, where the input genuinely is the previous layer's output after its nonlinearity. At the very first layer there is no earlier layer to have activated anything, so what the unified buffer actually holds is the network's raw input — pixel intensities, token embeddings, whatever the model consumes. The hardware cannot tell the difference and has no reason to — a vector of numbers looks the same to the MXU either way — which is why "activations" is used loosely for both cases, here and throughout this paper.

5.2 The programming model: how a TPU actually gets used

Anyone who has programmed a GPU will reasonably wonder whether a TPU works the same way: you write a kernel, launch it, and thousands of threads execute it in parallel. It doesn't, and being precise about the difference is the fastest way to understand what a TPU actually is.

Application developers normally do not write low-level TPU instructions directly. They express a model in a framework such as TensorFlow, JAX, or PyTorch — a description of layers, matrix multiplies, convolutions, and activation functions, with no reference to individual PEs. A compiler stack such as XLA [10] then lowers and maps the computation to the target TPU generation. The concrete instruction set and programming interface have evolved across TPU generations; the simplified TPU v1 picture here is intended to show the architectural separation between a high-level model, compiler-generated accelerator operations, and the fixed-function PE mesh rather than to specify a universal instruction set for every TPU.

The host and the accelerator then have a clean division of labor. At the conceptual level illustrated by TPU v1, the host and compiler arrange large operations and data movement, while the accelerator executes those operations without the host controlling individual PEs cycle by cycle. A single matrix operation can keep a large array busy for many cycles, whereas a general-purpose CPU instruction typically names a much smaller piece of work. This is worth contrasting with a CUDA kernel launch: a GPU is given a program that many SIMT threads execute in parallel, while a systolic matrix engine is given a large, pre-defined dataflow operation whose internal PEs do not fetch or decode their own instruction streams. Both are ways of offloading work to an accelerator, but the unit of programmability is different.

This also answers a natural question: is the TPU programmable, or is it fixed-function hardware? The honest answer depends on which level you look at. At the level of one PE inside the array, the answer is unambiguously fixed-function — Section 2 already established this: a PE has no instruction fetch, no program counter, and no way to do anything other than multiply, accumulate, and forward data, ever. At the level of the whole chip, the TPU is genuinely programmable, just not in the general-purpose sense: different models compile to different instruction sequences, so the chip is not hardwired to one fixed computation the way a video-decoder chip is hardwired to one codec, and its hardware interface is deliberately specialized around the operations supported by the accelerator rather than a general-purpose instruction set. This in-between position has a name: a *domain-specific architecture* [4] — not one hardwired function, and not a general-purpose processor, but a small, fixed instruction set built from a few large, complex operations.

5.3 What changed since 1978, and what didn't

The most durable ideas are the regular PE mesh, short local communication, synchronous data propagation, and aggressive spatial reuse. These principles remain recognizable from the original systolic proposal, even though modern implementations choose different dataflows and numerical formats. What changed around them is substantial: neural-network arithmetic often uses reduced precision; arrays are embedded in much larger memory, vector, and interconnect systems; and modern software stacks compile high-level tensor

programs into hardware-specific execution plans [7]. The architecture is therefore best understood as a continuity of principles rather than a literal preservation of one 1978 microarchitecture.

6 Beyond Google: The Idea Generalized

The resurgence of AI accelerators revived the broader architectural ideas behind systolic arrays: spatial matrix computation, regular data movement, and aggressive operand reuse. Some accelerators implement these ideas as explicit systolic arrays — for example, AWS Trainium and Inferentia2 expose an explicitly systolic 128×128 Tensor Engine [1]. Others use specialized matrix-multiply microarchitectures whose internal organization should not automatically be equated with a classical large systolic array. NVIDIA Tensor Cores, for example, provide small matrix-multiply-accumulate operations inside an otherwise SIMT GPU [9]; Apple's Neural Engine and other NPUs likewise pursue specialized, reuse-oriented tensor computation with generation-specific microarchitectures [2, 11]. Academic accelerators such as Eyeriss developed the distinct row-stationary mapping for convolution [3, 12]. The 1978 mesh therefore remains one important member of a broader family of spatial architectures rather than a label that should be applied indiscriminately to every modern matrix engine.

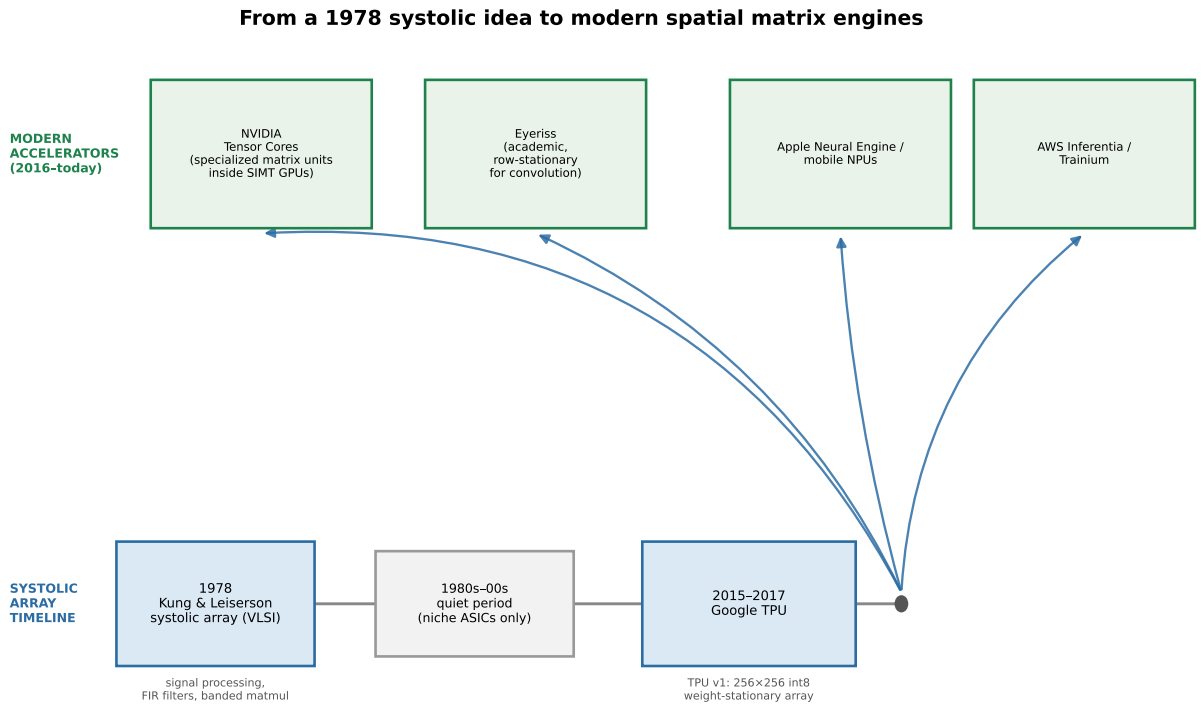


Figure 10: Two connected stories. The bottom row is a timeline: the 1978 Kung–Leiserson array, the quiet decades that followed, and the 2015–2017 TPU that brought systolic arrays back into mainstream AI hardware. The continuation then connects to modern accelerators that reuse related ideas of spatial matrix computation, regular data movement, and operand reuse; not all of them should be treated as identical implementations of one classical systolic dataflow.

7 Why the Idea Fits Today's Deep Learning Workloads

Many of the dominant compute kernels in deep learning are dense linear-algebra operations: matrix multiplies inside fully-connected and attention layers, together with convolutional computations that can often be mapped to dense matrix operations or related spatial dataflows. These regular workloads are especially well suited to systolic-style architectures for three reinforcing reasons.

7.1 High, and controllable, arithmetic intensity

Arithmetic intensity — FLOPs performed per byte moved across a chosen memory boundary — is one of the quantities used by the roofline model to relate computation to available memory bandwidth and peak compute throughput [13]. A poorly reused implementation of matrix multiplication can spend much of its time on the memory-bound slope. By keeping operands local and reusing them across many MACs, a systolic implementation can reduce memory traffic and thereby increase the effective arithmetic intensity seen at that boundary. This makes it more likely to approach the compute-bound regime, but it does not guarantee that every matrix multiplication is compute-bound: the outcome depends on matrix size, tiling, memory bandwidth, numerical format, and the surrounding memory hierarchy. A dedicated matrix accelerator can also provide a much higher peak-compute ceiling than a general-purpose core, as illustrated in Figure 11; the exact slopes, ceilings, and any crossover point depend on the specific implementations being compared, so the figure should be read as an illustrative example rather than a universal result.

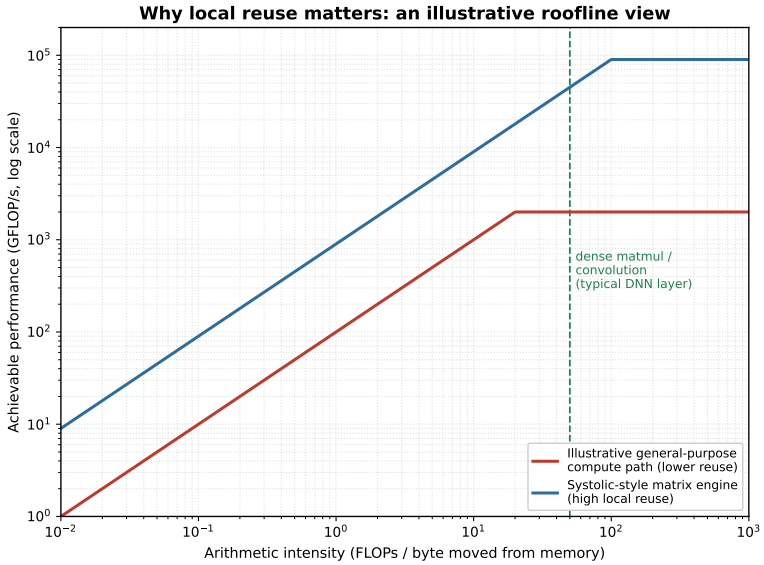


Figure 11: Illustrative roofline comparison. Local operand reuse can reduce memory traffic and increase effective arithmetic intensity, while a dedicated matrix engine can raise the peak-compute ceiling. Whether a particular matrix multiplication is memory- or compute-bound still depends on problem size, tiling, numerical format, bandwidth, and the memory hierarchy.

7.2 Energy: data movement, not arithmetic, dominates

Horowitz's widely cited energy estimates illustrate a central architectural fact: moving data through the memory hierarchy can cost orders of magnitude more energy than a simple arithmetic operation [5]. A systolic array's emphasis on local reuse and fewer memory round-trips is therefore also an energy-efficiency argument.

The exact energy ratios depend on process technology, data type, memory level, and implementation, but the qualitative imbalance between communication and arithmetic remains fundamental.

7.3 Control overhead amortized over huge, regular tensors

Many important deep-learning workloads involve large, regular tensor operations whose shapes can often be determined or specialized by the compiler ahead of execution. A compiler can then tile those operations to fit the array and map them onto repeated dataflow patterns. Dynamic shapes, sparsity, and irregular operations remain important exceptions, but this regularity is exactly what allows a fixed-function PE mesh to achieve high utilization — the same broad property that made convolution and FIR filtering attractive systolic targets in 1978.

7.4 Where the fit is imperfect

The same rigidity that gives systolic arrays their efficiency also creates known weaknesses, worth knowing honestly rather than glossing over:

- Utilization drops for matrices smaller than the array (e.g. a 64×64 weight matrix on a 256×256 MXU wastes most of the array, or requires batching several small matrices together).

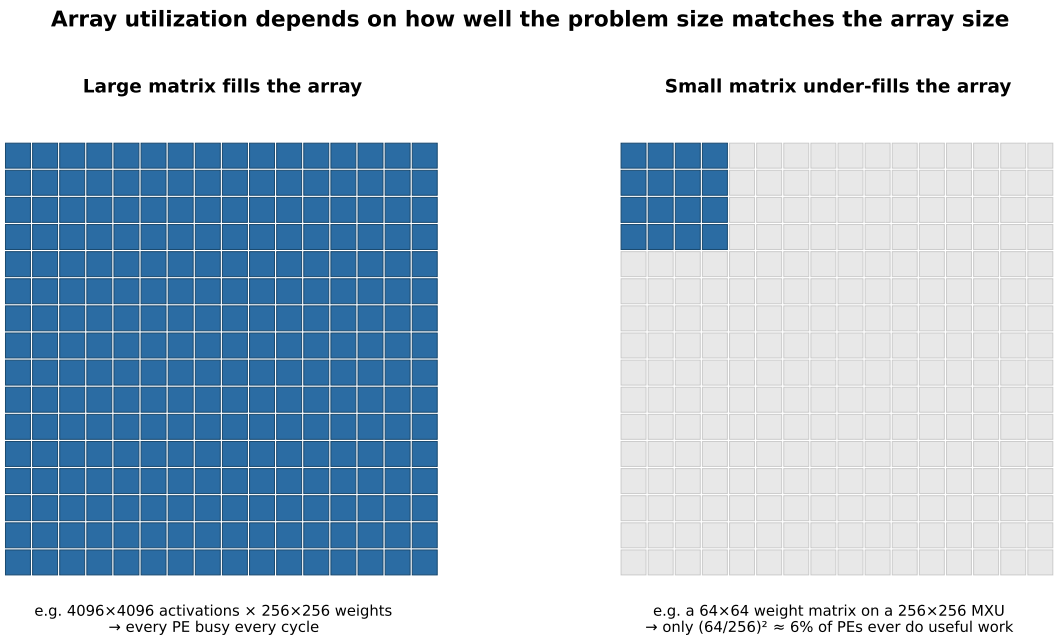


Figure 12: A matrix smaller than the array leaves most PEs idle every cycle — the same fixed mesh that gives perfect reuse on a well-matched problem gives poor utilization on a mismatched one.

- Weight-stationary designs pay a reload cost whenever the weight matrix changes faster than it can be reused — costly for workloads with rapidly changing weights, though not for the arrays and batches of most neural-network layers.
- Very sparse networks (many exact zeros) get little benefit unless the accelerator adds explicit sparsity support, since a plain systolic PE still spends a cycle multiplying by zero.

- Element-wise and irregular operations (softmax normalization, gather/scatter, control flow) are generally a poor fit for a systolic matrix mesh and are typically handled by other specialized units, vector/scalar hardware, or separate software paths.

8 Summary: 1978 vs. Today, Side by Side

Aspect	Kung & Leiserson systolic array (1978)	Google TPU-family matrix units (2016–today)
Target problem	Special-purpose VLSI for banded matrix–vector, convolution, FIR filtering	Dense matrix multiply / convolution inside neural networks
PE granularity	Simple fixed-function ALU, small local registers	Reduced-precision matrix arithmetic; numerical formats vary by generation
Array size (typical)	A few dozen cells (limited by 1970s–80s VLSI density)	256×256 on TPU v1; 128×128 MXUs on TPU v2/v3; later generations vary
Operand movement	Nearest-neighbour only, clocked lock-step	Within the matrix engine: regular local data movement; exact implementation varies by generation
Data stationarity	Problem-specific (often data streamed both ways)	Weight-stationary in the TPU v1 case study; later designs and workloads use additional mappings
Host coupling	Attached signal-processing co-processor	Accelerator integrated into a larger host/compiler/runtime system
Bottleneck it targets	I/O pins vs. computation (VLSI I/O-bound era)	Memory bandwidth / energy vs. computation (memory-wall era)
Why it faded / re-emerged	Faded as general CPUs rode Moore's law; re-emerged once DNN matmul workloads made data reuse dominant again	Adopted directly because DNN matmul reproduced the same reuse imbalance at datacenter scale

Table 2: Systolic arrays then and now.

References

[1] AWS Neuron Team. Trainium / Inferentia2 architecture guide for NKI. AWS Neuron Documentation, 2024. URL https://awsdocs-neuron.readthedocs-hosted.com/en/latest/nki/guides/architecture/trainium_inferentia2_arch.html. Describes the Tensor Engine as "built around a systolic array with 128 rows and 128 columns of processing elements".

[2] S. H. Bryngelson. Apple neural engine: Architecture, programming, and performance, 2026.

[3] Y.-H. Chen, J. Emer, and V. Sze. Eyeriss: A spatial architecture for energy-efficient dataflow for convolutional neural networks. In *Proceedings of the 43rd Annual International Symposium on Computer Architecture (ISCA)*, pages 367–379. IEEE Press, 2016. doi: 10.1109/ISCA.2016.40.

[4] J. L. Hennessy and D. A. Patterson. A new golden age for computer architecture, 2019. 2019 ACM A.M. Turing Award Lecture. See also J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed., Morgan Kaufmann, 2019, chapter on domain-specific architectures.

[5] M. Horowitz. 1.1 computing's energy problem (and what we can do about it). In *2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC)*, pages 10–14, 2014. doi: 10.1109/ISSCC.2014.6757323.

[6] N. P. Jouppi, C. Young, N. Patil, D. Patterson, G. Agrawal, R. Bajwa, S. Bates, S. Bhatia, N. Boden, A. Borchers, R. Boyle, P.-l. Cantin, C. Chao, C. Clark, J. Coriell, M. Daley, M. Dau, J. Dean, B. Gelb, T. V. Ghaemmaghami, R. Gottipati, W. Gulland, R. Hagmann, C. R. Ho, D. Hogberg, J. Hu, R. Hundt, D. Hurt, J. Ibarz, A. Jaffey, A. Jaworski, A. Kaplan, H. Khaitan, D. Killebrew, A. Koch, N. Kumar, S. Lacy, J. Laudon, J. Law, D. Le, C. Leary, Z. Liu, K. Lucke, A. Lundin, G. MacKean, A. Maggiore, M. Mahony, K. Miller, R. Nagarajan, R. Narayanaswami, R. Ni, K. Nix, T. Norrie, M. Omernick, N. Penukonda, A. Phelps, J. Ross, M. Ross, A. Salek, E. Samadiani, C. Severn, G. Sizikov, M. Snelham, J. Souter, D. Steinberg, A. Swing, M. Tan, G. Thorson, B. Tian, H. Toma, E. Tuttle, V. Vasudevan, R. Walter, W. Wang, E. Wilcox, and D. H. Yoon. In-datacenter performance analysis of a tensor processing unit. In *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA)*, pages 1–12. ACM, 2017. doi: 10.1145/3079856.3080246.

[7] N. P. Jouppi, G. Kurian, S. Li, P. Ma, R. Nagarajan, L. Nai, N. Patil, S. Subramanian, A. Swing, B. Towles, C. Young, X. Zhou, Z. Zhou, and D. A. Patterson. TPU v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings. In *Proceedings of the 50th Annual International Symposium on Computer Architecture (ISCA)*, pages 1–14. ACM, 2023. doi: 10.1145/3579371.3589350.

[8] H. T. Kung and C. E. Leiserson. Systolic arrays for VLSI. Technical report, Carnegie Mellon University, Department of Computer Science, 1978. Also published in H. T. Kung and C. E. Leiserson, "Systolic Arrays (for VLSI)," in C. Mead and L. Conway, *Introduction to VLSI Systems*, Addison-Wesley, 1980.

[9] S. Markidis, S. W. D. Chien, E. Laure, I. B. Peng, and J. S. Vetter. NVIDIA tensor core programmability, performance & precision. In *Proceedings of the 2018 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, pages 522–531. IEEE, 2018. doi: 10.1109/IPDPSW.2018.00091.

[10] OpenXLA Project. XLA: An open-source machine learning compiler for GPUs, CPUs, and ML accelerators. OpenXLA Project Documentation, 2024. URL https://openxla.org/xla.

[11] A. Reuther, P. Michaleas, M. Jones, V. Gadepally, S. Samsi, and J. Kepner. Survey of machine learning accelerators. In *2020 IEEE High Performance Extreme Computing Conference (HPEC)*, pages 1–12, 2020. doi: 10.1109/HPEC43674.2020.9286149.

[12] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer. *Efficient Processing of Deep Neural Networks*. Synthesis Lectures on Computer Architecture. Morgan & Claypool Publishers, 2020.

[13] S. Williams, A. Waterman, and D. Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009. doi: 10.1145/1498765.1498785.